

Enoncés TDs et TP / Angular (partie 2)

1. TD formulaire de login (template-driven)

Le framework Angular offre **deux possibilités** pour **effectuer des contrôles de saisies** au sein des **formulaires**:

- une **approche** dite **"template-driven"** où la plupart des **paramétrages** sont effectués du **côté ".html"** avec des syntaxes normalisées HTML5
- une **approche** dite **"model-driven"** où la plupart des **paramétrages** sont effectués du **côté ".ts"** (d'une manière très spécifique à angular)

Le formulaire de login sera basée sur l'approche "template-driven" à travers ce TD.

Un TP ultérieur permettra la mise en œuvre de l'approche model-driven/reactive-form .

Attention : sans ajout de FormsModule dans la partie imports: [] d'un composant ou de @NgModule , la gestion des formulaire ne fonctionne pas !!! .

Pour **bien organiser le code** , créer *dans src/app* un **nouveau "folder"** **src/app/common** et un **sous répertoire src/app/common/data**

Après s'être placé dans **src/app/common/data** (via cd), générer une nouvelle classe de données **Login** via la commande **ng g class login**

Au sein de la classe **Login**, **ajouter 3 attributs** de type **string** nommés **username** , **password** et **roles** .

src/app/common/data/login.ts

```
export class Login {
  constructor(
    public username : string = "",
    public password : string = "",
    public roles : string = "" ){}
}
```

Au sein de la classe **LoginComponent** , **ajouter un attribut login** de type **Login** et ayant **= new Login()** comme valeur par défaut.

Ajouter également dans LoginComponent (coté .ts):

- une propriété **message** de type **string**
- une méthode **onLogin()** qui dans un premier temps se contentera de mettre à jour le message en fonction des valeurs saisies cotés .html :

src/app/login/login.component.ts

```
import { Component, OnInit } from '@angular/core';
import { Login } from '../common/data/login'

@Component({
  selector: 'app-login',
  imports: [FormsModule],
  templateUrl: './login.component.html',
  styleUrls: ['./login.component.scss']
})
export class LoginComponent {
```

```

public login : Login = new Login();
public message /* :string */ = "";
public onLogin(){
    this.message = "donnees saisies = " + JSON.stringify(this.login);
}

constructor() { }
}

```

Dans **LoginComponent.html** ajouter de quoi saisir username, password et roles avec des [(ngModel)] paramétrés sur **login.username** etc :

Ajouter également un bouton poussoir **login** déclenchant **onLogin()**:

src/app/login/login.component.html

```

<h3>login</h3>
<label>username:</label>
  <input [(ngModel)]="login.username" /> <br/>
<label>password:</label>
  <input [(ngModel)]="login.password" /> <br/>
<label>roles:</label>
  <input [(ngModel)]="login.roles" /> <br/>
<input type="button" value="login" (click)="onLogin()" /> <br/>
message: {{message}}

```

Lancer **ng serve** dans un terminal libre de *Visual Studio Code*

Visualiser un résultat de ce type dans un navigateur internet (<http://localhost:4200>):

login

username:

password:

roles:

message: donnees saisies = {"username":"user1","password":"pwduser1",

NB: Le framework **angular**, analyse les **attributs/propriétés** (selon syntaxes normalisées **HTML5**) des **éléments d'un formulaire** (du coté template HTML) et **affecte automatiquement certaines classes de styles** selon les **circonstances suivantes** :

Etat	flag (booléen)	Css class si true	Css class si false
Champ visité (souris entrée et sortie)	touched	ng-touched	ng-untouched
Valeur du champ modifiée	dirty	ng-dirty	ng-pristine
Valeur du champ valide	valid	ng-valid	ng-invalid

NB:

- Le framework **angular** **active ou désactive automatiquement** les classes de styles ng-valid , ng-invalid , etc dont les noms sont prédéfinis (convenus à l'avance) au niveau des champs d'un formulaire .

- C'est néanmoins au **développeur** que revient le soin d'**associer une mise en forme souhaitée** à ces **classes de styles** dans le fichier global **src/styles.css** ou bien dans le fichier **...component.css** .

A faire en TD: ajouter les lignes suivantes dans **src/styles.css** ou ailleurs:

```
input.ng-valid {  
  border-left: 5px solid #42A948; /* green */  
}  
  
input.ng-invalid {  
  border-left: 5px solid #a94442; /* red */  
}
```

Rappels des principales contraintes de saisies d'HTML5:

- **required** : le champ est requis (valeur obligatoire)
- **minlength="3"** : il faut saisir au moins 3 caractères
- **pattern="^[a-zA-Z].+"** : la valeur saisie doit correspondre à une expression régulière (ici ça doit commencer par un caractère alphabétique puis contenir au moins un autre caractère quelconque)
- ...

Ajouter les contraintes de saisies suivantes dans **src/app/login/login.component.html**

```
<h3>login</h3>  
<label>username:</label>  
  <input [(ngModel)]="login.username" required pattern="^[a-zA-Z].+" />  
  <br/>  
<label>password:</label>  
  <input [(ngModel)]="login.password" required minlength="3"/>  
  <br/>  
<label>roles:</label>  
  <input [(ngModel)]="login.roles" required />  
  <br/>  
<input type="button" value="login" (click)="onLogin()" />  
  <br/>  
message: {{message}}
```

Tester le nouveau comportement avec un navigateur:

login

username:

password:

roles:

message:

login

username:

password:

roles:

On va encore **peaufiner le formulaire** en faisant en sorte que **le bouton "login" reste grisé (disabled) tant que l'ensemble des champs de saisies ne sont pas tous valides.**

Il faut pour cela , englober les champs du formulaire via `<form #formXy="ngForm" >` et `</form>`

Le principal intérêt d'écrire explicitement `<form #formXy="ngForm" >` est de pouvoir écrire plus bas :

```
<input type="button ou submit" value="..." (click)="..."
      [disabled]="!formXy.form.valid" />
<!-- déclencheur d'action grisé tant que formulaire pas globalement valide -->
```

NB: Chaque champ du formulaire doit absolument avoir un nom de renseigné (via **name="..."**) dès qu'il se trouve encadré par `<form ...>` et `</form>`. Sinon : erreur du côté ng serve (ou du côté console du navigateur) .

A faire en TD:

Ajouter tous les éléments vus précédemment dans `src/app/login/login.component.html`

```
<h3>login</h3>
<form #formLogin="ngForm">
  <label>username:</label>
  <input [(ngModel)]="login.username" name="username"
    required pattern="[a-zA-Z].+" />
  <br/>
  <label>password:</label>
  <input [(ngModel)]="login.password" name="password"
    type="password" required minlength="3"/>
  <br/>
  <label>roles:</label>
  <input [(ngModel)]="login.roles" name="roles" required />
  <br/>
  <input type="button" value="login" (click)="onLogin()"
    [disabled]="!formLogin.form.valid" />
  <br/>
</form>
message: {{message}}
```

Bien tester bien le tout (via <http://localhost:4200>):

login

username: a
password:
roles: admin

message:

login

username: admin1
password:
roles: admin

message: donnees saisies = {"username":"admin1","password":"pwdadm

2. TD facultatif : intégration de bootstrap-css

2.1. Intégration de bootstrap-css dans angular

Lancer les commandes suivantes au sein d'un terminal (en étant placé à la racine de l'application angular: là où est package.json):

```
npm install --save bootstrap  
npm install --save bootstrap-icons
```

Visualiser les dépendances ajoutées dans package.json:

```
...  
"bootstrap": "^5.3.3",  
"bootstrap-icons": "^1.11.3",  
...
```

Ajouter les chemins qui mènent à `./node_modules/bootstrap/dist/css/bootstrap.min.css` et autres dans la partie **styles** de la partie **architect/build/options** de **angular.json** (près de la ligne 31):

```
"styles": [
  "src/styles.css" ,
  "node_modules/bootstrap/dist/css/bootstrap.min.css",
  "node_modules/bootstrap-icons/font/bootstrap-icons.css"
]
```

Attention à ne pas introduire d'erreur en plaçant mal une virgule .

2.2. Test de l'intégration bootstrap/angular:

Effectuer des ajouts de classes "bootstrap" et "fontawesome" dans src/app/login/login.component.html en adaptant si besoin cet exemple:

```
...
<button class="btn btn-primary" (click)="onLogin()"
  [disabled]="!formLogin.form.valid" >
  login <span class="badge badge-primary">
    <i class="bi bi-person"></i>
  </span>
</button>
...
```

Arrêter (si besoin) et relancer **ng serve** et visualiser les résultats via <http://localhost:4200>



2.3. Ajustements "container-fluid" ou "container":

NB: Le fait d'intégrer "bootstrap css" à l'application Angular fait que malheureusement les marges sur les cotés sont peut être disparues.

Pour les rétablir on peut ajouter **<div class="container-fluid">** et **</div>** dans le composant principal src/app/app.component.html :

```
<div class="container-fluid">
  <app-header></app-header>
  ...
  <app-footer></app-footer>
</div>
```

2.4. Formulaire responsive avec champs bien alignés:

On va maintenant terminer ce TD en **ajoutant** les classes de styles de bootstrap css (ex: "row", "col-sm-4", ...) au sein de **login.component.html** pour obtenir un formulaire **responsive** avec des **champs bien alignés**:

```
<h3>login</h3>
<form #formLogin="ngForm">
  <div class="form-group row">
```

```

<label class="col-sm-4">username:</label>
<div class="col-sm-8">
  <input [(ngModel)]="login.username" name="username"
    class="form-control" required pattern="[a-zA-Z].+" />
</div>
</div>
<div class="form-group row">
  <label class="col-sm-4">password:</label>
  <div class="col-sm-8">
    <input [(ngModel)]="login.password" name="password"
      type="password" class="form-control" required minlength="3"/>
  </div>
</div>
<div class="form-group row">
  <label class="col-sm-4">roles:</label>
  <div class="col-sm-8">
    <input [(ngModel)]="login.roles" class="form-control"
      name="roles" required />
  </div>
</div>
<div class="form-group row">
  <div class="col-sm-8 offset-4">
    <button class="btn btn-primary" (click)="onLogin()"
      [disabled]="!formLogin.form.valid" > login </button>
  </div>
</div>
</form>
message: {{ message }}

```

```

input.ng-valid.form-control { width:100%; margin-top: 2px; border-left: 5px solid #42A948; }
input.ng-invalid.form-control { width:100%; margin-top: 2px ; border-left: 5px solid #a94442; }

```

username:	user1
password:	pwd1
roles:	user
submit/login	

3. TD alternatif facultatif : custom css (flexbox)

Ne pas utiliser bootstrap-css mais se construire des classes de styles basés sur les "flexbox" de manière à obtenir un bon comportement "responsive" au niveau des formulaires.
On pourra facultativement s'appuyer sur des directives personnalisées.

Exemples :

styles.scss

```

.f-form-group-wrap {
  display: flex;
  flex-flow: row wrap;
}

```

```
label.f-align {
  width: 12em;
  display: inline-block;
  margin-left: 0.3em;
  flex: 0 0 12em;
}
```

```
.f-align {
  margin-left: 0.3em;
  flex: 1 1 auto;
}
```

```
.f-max-size {
  width: 98%;
}
```

```
<form #formLogin="ngForm">
  <div class="f-form-group-wrap">
    <label class="f-align">username:</label>
    <div class="f-align">
      <input [(ngModel)]="login.username" class="f-max-size"
        name="username" required pattern="^[a-zA-Z].+"/>
    </div>
  </div>
  ...
```

4. Tp (facultatif) reactive-form

- Générer un nouveau composant "reservation" via ng component ... *--type component*
- ajouter ce nouveau composant dans la liste des routes du app-routing.module.ts ou app.routes.ts et au sein du menu de navigation (dans header.component.html)

Après s'être placé dans src/app/common/data (via cd), générer une nouvelle classe de données *Login* via la commande **ng g class reservation**

Coder la classe **Reservation** comme ceci :

src/app/common/data/reservation.ts

```
export class Reservation {
  constructor(
    public firstName : string ="prenom",
    public lastName : string ="Nom",
    public telephone : string ="0605040302",
    public email : string ="prenom.Nom@xyz.com" ,
    public dateTime : Date = new Date()){}
```

Coder les cotés .ts et .html de ReservationComponent

reservation

firstName:		firstName is required
lastName:	Nom	
telephone:	060504030	telephone.minLength=10
email:	prenom.Nomxyz.com	email should be a valid email with ...@...
date:	2024-10-17	
time:	11:0	time HH:MM

5. TP input() sur composant header

Ajouter (via **input()**) un titre paramétrable au sein du composant HeaderComponent

Afficher la valeur de `{{titre()}}` dans `header.component.html`

Au sein de `app.component.html`, donner une valeur fixe au titre paramétrable de `<app-header>`

Tester le tout

Donner ensuite une valeur dynamique au titre paramétrable de `<app-header>` en demandant une copie de la valeur de `AppComponent.title`.

Tester le nouveau comportement.

`my-app` `welcome` `basic`

welcome



footer , date=13/09/2024

6. TP simpleSelector via output() puis model()

- (idéalement dans **src/app/common/component**) générer un nouveau composant réutilisable appelé "**SimpleSelector**" et intégrer le quelque part (par exemple au sein de **basic.component.html**)
- coder une première version de ce composant (avec `input()` et `output()`) de manière à ce que l'on puisse paramétrer :
 - * un *title* variable
 - * une liste variable *values* de valeurs (de type `string[]`) à choisiret que l'on puisse récupérer la valeur choisie sous la forme d'un événement *choix*

Exemple de look souhaité pour le composant "SimpleSelector" ici en fond gris:

choix couleur

- red
- **green**
- blue

couleurChoisie = green

Exemple d'utilisation souhaitée de ce composant depuis un parent :

```
<app-simple-selector  
  title="choix couleur"  
  [values]="[ 'red' , 'green', 'blue' ]"  
  (choix)="onChoixCouleur(.....)"></app-simple-selector> <br/>  
couleurChoisie = {{couleurChoisie}}
```

Suite facultative du Tp :

- Sauvegarder des copies de `simple-selector.component.html` et `.ts` (ex : `.v1.txt`)
- Puis au sein de `simple-selector.component.ts` , transformer *choix* de `output()` en **model()** de manière à ce que le parent puisse (s'il en a envie) fixer un choix par défaut (ex : `'green'`)
- rendre tout cela cohérent et tester le nouveau comportement (choix par défaut paramétrable) dans une version adaptée du composant parent (après une éventuelle copie de l'ancienne version en commentaires)

7. TD projection dans composant réutilisable

Préparer l'arborescence **src/app/common/component** et se placer dans ce répertoire .

```
ng g component toggle-panel --type component
```

Coder le contenu du composant **toggle-panel** en s'inspirant du code du support de cours (chapitre "composants et modules / approfondissements" et sous partie "projection d'éléments imbriquées"). avec `<ng-content></ng-content>` et avec ou sans l'utilisation de bootstrap-css

Ajuster le contenu de `basic.component.html` de façon à ce que les composants "calculatrice" et "tva" soient imbriqués dans des panneaux déployables/récontractables :

```
<app-toggle-panel title="calculatrice">
  <app-calculatrice></app-calculatrice>
</app-toggle-panel>

<app-toggle-panel title="calcul tva">
  <app-tva></app-tva>
</app-toggle-panel>
```

calculatrice ↑

calculatrice

a: 5

b: 6

+ - * /

res= 11

Zone à survoler à la souris .
x=0 , y=0

calcul tva ↓

8. TD (facultatif) : onglet de @angular/material

Intégration de l'extension @angular/material au sein du projet angular :

ng add @angular/material

>>> Choisir un des thèmes proposés et accepter les animations

Importation du module "MatTabsModule" (onglets de material) :

Ajouter dans `src/app/app.module.ts` ou bien dans `src/app/basic/basic.component.ts`

```
import {MatTabsModule} from '@angular/material/tabs';
et
imports: [
  ...,
  MatTabsModule,
],...
```

Utilisation possible au sein de `basic.component.html` :

```
<mat-tab-group>
  <mat-tab label="calculatrice">
    <app-calculatrice></app-calculatrice>
  </mat-tab>
```

```
<mat-tab label="calcul tva">
  <app-tva></app-tva>
</mat-tab>
</mat-tab-group>
<hr/>
```

calculatrice

calcul tva

calcul tva

ht: 0

tauxTva(%): 20% ▼

Un redémarrage de ng serve sera peut être nécessaire ...

9. TP (facultatif) directive "highlight-over"

Au sein du composant "welcome" , placer une série de 3 ou 4 petites images.

On va maintenant coder et utiliser une **directive personnalisée "border-over"** qui va automatiquement mettre en évidence une des images survolées (en ajoutant une bordure par défaut rouge ou bien d'une couleur paramétrable).

Indications :

créer si besoin le sous répertoire src/app/**common/directive**
ng g directive border-over